Load Balancer para Modelos de IA

Artur Fernandes Braga de Menezes[1], **Artur Rizzi Martinho**[1], **Felipe Lacerda Tertuliano**[1], **Gabriel Alves da Silva Diógenes**[1], **Rafael Mortimer Colares**[1]

[1]Pontifícia Universidade Católica de Minas Gerais (PUC Minas)
30.535-901 - Belo Horizonte - MG - Brazil

Abstract. *The evolution of Artificial Intelligence (AI) models, especially Large Language Models (LLMs), has significantly expanded the capacity of computational systems to perform tasks such as text generation and comprehension, programming, and information processing. However, the increased capability of these models is also associated with greater demand for computational resources during their use. Moreover, different applications may present distinct requirements for processing, latency, and resource usage, making efficient request management a relevant challenge for systems that provide AI models at scale. Studies on LLM serving systems highlight precisely the heterogeneity of requests and the need for efficient mechanisms to distribute and schedule their execution. In this context, this work proposes the study and development of a Load Balancer for AI models, capable of distributing requests among different models according to their characteristics and resource availability. The proposed approach investigates the impacts of this strategy on computational resource usage, response time, and processing cost.*

Resumo. *A evolução dos modelos de Inteligência Artificial (IA), especialmente dos Grandes Modelos de Linguagem (Large Language Models — LLMs), tem ampliado significativamente a capacidade dos sistemas computacionais de realizar tarefas como geração e compreensão de texto, programação e processamento de informações. Entretanto, o aumento da capacidade desses modelos também está associado a uma maior demanda por recursos computacionais durante sua utilização. Além disso, diferentes aplicações podem apresentar requisitos distintos de processamento, latência e utilização de recursos, tornando o gerenciamento eficiente das requisições um desafio relevante para sistemas que disponibilizam modelos de IA em larga escala. Estudos sobre sistemas de serving de LLMs destacam justamente a heterogeneidade das requisições e a necessidade de mecanismos eficientes para distribuir e agendar sua execução. Nesse contexto, este trabalho propõe o estudo e desenvolvimento de um Load Balancer voltado para modelos de IA, capaz de distribuir as requisições entre diferentes modelos de acordo com suas características e com a disponibilidade dos recursos. A abordagem proposta investiga os possíveis impactos dessa estratégia sobre a utilização dos recursos computacionais, o tempo de resposta e o custo de processamento.*

1. Introdução

A evolução dos modelos de Inteligência Artificial (IA), especialmente dos Grandes Modelos de Linguagem (Large Language Models — LLMs), tem ampliado significativamente

a capacidade dos sistemas computacionais de realizar tarefas como geração e compreensão de texto, programação e processamento de informações. Entretanto, o aumento da capacidade desses modelos também está associado a uma maior demanda por recursos computacionais durante sua utilização. Além disso, diferentes aplicações podem apresentar requisitos distintos de processamento, latência e utilização de recursos, tornando o gerenciamento eficiente das requisições um desafio relevante para sistemas que disponibilizam modelos de IA em larga escala. Estudos sobre sistemas de serving de LLMs destacam justamente a heterogeneidade das requisições e a necessidade de mecanismos eficientes para distribuir e agendar sua execução [Kossmann et al. 2024, Sun et al. 2024].

Nesse contexto, utilizar um único modelo de grande porte para atender diferentes tipos de requisições pode resultar em uma utilização pouco eficiente dos recursos disponíveis, principalmente quando determinadas tarefas poderiam ser executadas por modelos menores ou especializados. O problema também se estende à distribuição das requisições entre diferentes instâncias, uma vez que variações na demanda podem provocar sobrecarga em determinados recursos enquanto outros permanecem subutilizados. Trabalhos recentes demonstram que estratégias de balanceamento e escalonamento podem melhorar a utilização da infraestrutura disponível. O SkyWalker, por exemplo, propõe um mecanismo de balanceamento de carga entre diferentes regiões para inferência de LLMs, buscando utilizar os recursos de forma mais eficiente e reduzir custos de serving [Xia et al. 2025]. De forma semelhante, o Llumnix explora o reagendamento dinâmico de requisições entre múltiplas instâncias de modelos para lidar com diferentes requisitos de recursos e latência [Sun et al. 2024].

Diante desse cenário, este trabalho propõe o estudo e desenvolvimento de um Load Balancer voltado para modelos de IA, capaz de distribuir as requisições entre diferentes modelos de acordo com suas características e com a disponibilidade dos recursos. A utilização de múltiplos modelos pode permitir que diferentes tipos de solicitações sejam direcionados para alternativas mais adequadas, evitando o uso desnecessário de modelos de maior porte quando modelos mais simples forem suficientes para determinada tarefa. Dessa forma, busca-se investigar os possíveis impactos dessa abordagem sobre aspectos como utilização dos recursos computacionais, tempo de resposta e custo de processamento. A literatura demonstra que mecanismos de balanceamento e escalonamento aplicados ao serving de LLMs podem proporcionar ganhos de desempenho e redução de custos, reforçando a relevância de investigar estratégias semelhantes em arquiteturas compostas por múltiplos modelos [Kossmann et al. 2024, Xia et al. 2025, Sun et al. 2024]. O protótipo construído materializa essa proposta em uma CLI única, que cadastra modelos locais e remotos, classifica os prompts e despacha as requisições pelo load balancer.

2. Revisão bibliográfica

Sistemas de inferência de LLMs não tratam todas as requisições da mesma forma. Há prompts curtos e longos, respostas de poucos tokens e de milhares, picos de demanda e períodos ociosos. Por isso, o load balancer deixou de ser um detalhe de infraestrutura e passou a definir latência, custo e uso de GPU. Os cinco trabalhos a seguir discutem justamente esse componente: a quem enviar cada requisição. Nenhum deles, porém, escolhe entre modelos de portes diferentes, que é o recorte deste projeto.

Kossmann et al. descrevem a arquitetura mais comum desses sistemas em dois

níveis [Kossmann et al. 2024]. Na entrada, o load balancer escolhe o servidor que guarda uma réplica do modelo. Dentro do servidor, um scheduler decide se a requisição espera na fila, entra no lote de execução ou é interrompida. Na prática, muitos serviços ainda usam round-robin, escolha aleatória ou *power of two choices*. Os autores propõem o SAL (*Server-Aware Load Balancer*), que estima a carga de cada máquina a partir dos tokens ainda na fila e da memória livre, e envia o pedido para a menos carregada. O SAL é simples de colocar no lugar da política atual e reduz o tempo de geração em relação a balanceadores genéricos. O artigo liga redes, ou seja, qual servidor recebe o pedido, a sistemas operacionais, isto é, como a fila e a preempção funcionam na GPU.

Jain et al. mostram que balancear entre várias instâncias do mesmo modelo pesa mais na latência do que só ajustar o scheduler de uma GPU [Jain et al. 2025]. Round-robin e políticas parecidas tratam a inferência como um job único e ignoram as duas fases do processamento: o *prefill*, mais pesado em computação, e o *decode*, mais pesado em memória. Misturar esses perfis na mesma instância atrasa todo mundo. O load balancer deles prevê o tamanho da resposta e o efeito de misturar cargas. Em quatro instâncias, a latência média cai 11,43% em 2.000 requisições. A limitação é clara e importa para este trabalho: todas as instâncias servem o mesmo modelo. Não há escolha entre um modelo pequeno e um grande.

O Llumnix parte de outra observação: o tempo de serviço e o uso de memória de uma requisição de LLM não se conhecem de antemão [Sun et al. 2024]. O KV cache cresce a cada token, as prioridades mudam e um único despacho no início não dá conta. O sistema reagenda pedidos em execução, migrando a requisição e o estado na GPU de uma instância para outra, como a troca de contexto entre núcleos no sistema operacional. Isso reduz interferência, fragmentação de memória e violação de SLO. Num cluster de 16 GPUs, a latência P99 do primeiro token cai até $15\times$ frente a um scheduler clássico. O ganho de custo chega a 36% para a mesma latência de cauda. Aqui o load balancing aparece como problema de sistemas operacionais: fila, isolamento, preempção e migração de estado.

O SkyWalker leva o mesmo problema para a rede geográfica [Xia et al. 2025]. Provedores reservam GPU em cada região para o pico local e desperdiçam capacidade fora do horário de pico. Um balanceador único no centro do mapa barateia o serviço, mas vira gargalo e ponto único de falha. A proposta coloca um load balancer por região e só desvia tráfego quando uma região satura. Round-robin e *least-load* falham porque o tempo de uma inferência varia de segundos a dezenas de segundos, e porque pedidos com o mesmo prefixo deveriam cair na GPU que já tem o KV cache. Em cargas reais, o SkyWalker aumenta a vazão em $1,12$ a $2,06\times$ e reduz a latência em $1,74$ a $6,30\times$ diante de load balancers existentes, com cerca de 25% menos custo. Para este projeto, o recorte multi-região fica fora do protótipo. O que permanece é a lição de rede: o despacho precisa olhar latência, vazão e estado dos backends, não só um rodízio cego.

O DualMap trata o conflito entre duas políticas que os artigos anteriores encostam, mas não unem de fato [Yuan et al. 2026]. Mandar pedidos com o mesmo prefixo para a mesma máquina melhora o cache e o tempo até o primeiro token, mas cria pontos quentes. Espalhar pela menor carga equilibra o cluster e quebra o cache. A solução mapeia cada requisição a duas instâncias candidatas, a ideia clássica de *power of two choices*, e escolhe a melhor no momento. Se o SLO de TTFT ameaça estourar, o sistema deixa o cache em

segundo plano e passa a olhar carga. Se uma máquina esquenta, migra pedidos para a instância reserva. Sobre o vLLM, a capacidade efetiva de requisições sobe até $2,25\times$ com o mesmo SLO. O artigo une redes, no despacho e na elasticidade, e arquitetura, no KV cache e na ocupação de GPU.

Os cinco trabalhos concordam em três pontos. Load balancing de LLM não se comporta como o de um site comum: o serviço é longo, tem duas fases e o tamanho da resposta não é conhecido. O balanceador e o scheduler interno são camadas diferentes; otimizar só uma não basta [Kossmann et al. 2024, Jain et al. 2025]. Memória, KV cache, fragmentação e localidade de prefixo entram na decisão [Sun et al. 2024, Xia et al. 2025, Yuan et al. 2026].

A lacuna que este trabalho explora está no que todos eles deixam de fora. Os cinco balanceiam réplicas do mesmo modelo, ou do mesmo serviço, e perguntam apenas *qual instância* deve atender o pedido. Nenhum classifica a complexidade da consulta para escolher entre um modelo leve e um modelo maior, nem junta essa escolha com o estado das filas. O load balancer proposto aqui continua sendo um despachante de camada de aplicação, mas o conjunto de destinos passa a depender do tipo de prompt, e a instância escolhida, da carga observada.

3. Metodologia de Desenvolvimento e Avaliação

A metodologia adotada neste trabalho organiza-se em torno de ciclos semanais de desenvolvimento e avaliação, conforme apresentado na Tabela 1. Se espera ter até a sexta semana um código completo capaz de atender nossos requisitos e ser utilizado para as medições, a partir disso o foco do trabalho se vira para coleta de resultados, melhorias, redação do artigo e apresentações.

Quanto ao método de avaliação e validação foi escolhida a medição em ambiente controlado complementada pela analise dos resultados, isso porque o desempenho de um LoadBalancer para modelos de IA depende de fatores dinâmicos do ambiente que não podem ser capturados por simulações. A validação será conduzida por meio de experimentos comparativos entre dois cenários: (i) *baseline*, no qual todas as requisições são direcionadas ao modelo de maior porte; e (ii) *proposta*, na qual o LoadBalancer classifica o prompt e o encaminha ao modelo mais adequado. Serão coletadas métricas de latência média, P95 e P99, além de uso de RAM e GPU por máquina, a análise estatística dos dados incluirá identificação de outliers e geração de gráficos comparativos.

O ambiente experimental é composto por uma máquina executando o Ollama com modelos leves (Phi-3.5-mini, Qwen2.5-Coder-0.5B e AILO-152M-v2). O LoadBalancer é implementado em Python com FastAPI, utiliza `httpx` para comunicação com os servidores de inferência e mantém um catálogo SQLite de modelos, com classe, status e URL por backend. Uma CLI unificada concentra o cadastro dos modelos, a classificação, a inferência, a execução do serviço e a análise dos experimentos. Ajustes podem envolver a inclusão de novas métricas, a alteração do conjunto de modelos avaliados ou a adoção complementar de técnicas de simulação, caso se mostrem necessárias para validar aspectos específicos do sistema.

Tabela 1. Cronograma de atividades do projeto.

Semana	Atividades principais
1	Configuração do ambiente (Ollama, modelos leves), levantamento bibliográfico inicial.
2	Desenvolvimento do classificador de prompts, implementação da versão inicial do LoadBalancer.
3 (Sprint 2)	Detalhamento da implementação, coleta de resultados preliminares, preparação e apresentação oral do Sprint 2.
4	Refinamento do classificador, implementação de métricas de monitoramento, escrita da metodologia.
5	Expansão do conjunto de testes, execução de experimentos comparativos, análise estatística dos resultados.
6	Otimização do LoadBalancer, testes automatizados de roteamento, escrita da seção de resultados.
7	Comparação com trabalhos relacionados, redação da seção comparativa, preparação de material visual para o pitch.
8	Redação das conclusões, revisão e refinamento do artigo, roteiro do pitch comercial.
9 (Sprint 3)	Gravação e edição do pitch comercial, preparação e apresentação oral final.
10	Submissão final do artigo, documentação do projeto e entrega dos materiais de apresentação.

4. Desenvolvimento e Implementação

O LoadBalancer desenvolvido é um serviço HTTP em Python, construído sobre FastAPI, que fica entre o cliente e as instâncias do Ollama. Ele recebe a requisição, decide qual dos modelos cadastrados deve atendê-la e devolve a resposta junto com os metadados da decisão. A implementação foi organizada em quatro camadas — apresentação, aplicação, domínio e dados —, com uma única porta de entrada, a CLI unificada. O núcleo de aplicação mantém três responsabilidades que se encaixam em sequência: a classificação do prompt, que define o destino; o despacho para o Ollama, que executa a inferência e mede a latência; e a resiliência, que trata falhas sem comprometer a resposta. As subseções a seguir descrevem o classificador, o despacho e, por fim, o catálogo e a CLI que conectam essas peças.

4.1. Classificador de prompts

A primeira responsabilidade do serviço é decidir, antes de qualquer chamada ao Ollama, qual dos modelos cadastrados atende cada requisição. Essa decisão é tomada pela função pura `classificar(prompt)`, em `src/classificador.py`, que não faz entrada e saída nem depende de estado: a mesma entrada produz sempre a mesma saída. O texto é convertido para minúsculas e tem os acentos removidos; em seguida, as regras da Tabela 2 são avaliadas em ordem, e a primeira que se aplica define a classe. A tabela também registra a tag padrão de cada classe, usada quando o catálogo ainda não tem um modelo ativo para ela.

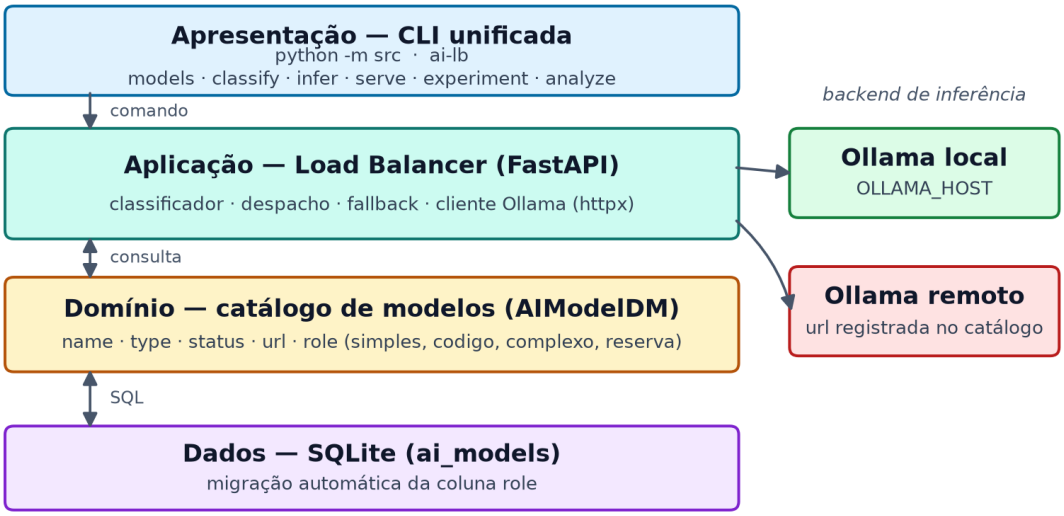


Figura 1. Arquitetura em camadas do projeto. A CLI unificada é a porta de entrada; o load balancer consulta o catálogo de modelos para resolver classe e URL de cada destino e despacha para o Ollama local ou remoto.

A escolha por regras, e não por aprendizado, foi deliberada nesta Sprint. Como o objetivo é atribuir diferenças de latência ao roteamento, a decisão precisa ser estável entre rodadas e explicável no log. Um classificador por aprendizado introduziria variação na própria variável que se quer medir.

Tabela 2. Regras do classificador, em ordem de prioridade

#	Condição	Classe	Modelo
1	Prompt com mais de 120 palavras	complexo	Phi-3.5-mini
2	Sinal de código	código	Qwen2.5-Coder-0.5B
3	Sinal de raciocínio	complexo	Phi-3.5-mini
4	Até 25 palavras, sem sinais	simples	AILO-152M-v2
5	Demais casos (ambíguo)	complexo	Phi-3.5-mini

Os sinais de código são expressões regulares que identificam blocos delimitados por três crases, nomes de linguagens (Python, Java, SQL etc.), termos como “função”, “implemente”, “bug” e “compilar”, e trechos de sintaxe como `def nome(` ou `import`. Os sinais de raciocínio identificam pedidos como “explique”, “por que”, “compare”, “analise”, “passo a passo” e “resolva”. O limite de 120 palavras vem primeiro porque um pedido longo exige contexto que os modelos de 0,5B e 152M parâmetros tendem a não sustentar, qualquer que seja o tema.

Há dois casos ambíguos tratados de forma explícita. Quando o prompt tem sinais de código e de raciocínio (por exemplo, “explique por que este código Python é lento”), a regra de código tem prioridade, porque o modelo especializado conhece melhor o domínio, e o motivo registrado menciona os dois sinais. Quando o prompt tem tamanho médio e nenhum sinal, a escolha é o Phi-3.5-mini: errar para o modelo maior aumenta a latência, mas não compromete a resposta, ao passo que errar para o modelo menor pode

comprometê-la.

A função devolve a classe, o modelo escolhido, o motivo em texto e a contagem de palavras. Esses campos são a entrada da etapa seguinte: é a classe devolvida aqui que o despacho usa para resolver o destino no catálogo e, com ele, a tag e a URL do backend. A função `classificar_e_registrar` grava os mesmos campos no log de cada requisição, junto com o identificador do prompt. Sem esse registro, a latência medida em um cenário não teria explicação: é ele que permite separar, na análise, as requisições atendidas por cada modelo e verificar se o classificador acertou a classe esperada do conjunto de prompts. A função tem testes automatizados para os três casos principais, para o pedido longo e para os dois casos ambíguos.

Como limitação, regras por palavra-chave produzem falsos positivos ("o que é uma API?" é factual, mas contém um sinal de código) e dependem do idioma. O refinamento do classificador, com novas categorias e a avaliação de acurácia sobre o conjunto rotulado, está previsto para a semana 4 do cronograma.

4.2. Arquitetura e despacho

Definido o destino pelo classificador, a segunda responsabilidade do serviço é executar a inferência e medir quanto ela custou. A biblioteca FastAPI publica o endpoint `POST /infer`, cujo corpo traz o texto do prompt e, quando a campanha precisa repetir o mesmo item, um identificador fixo. O endpoint `POST /v1/chat/completions` aceita o formato usual de conversa, extrai a última mensagem do usuário e segue o mesmo caminho. Nos dois casos, um eventual nome de modelo enviado pelo cliente é ignorado: o destino sai do classificador e do catálogo descritos nas subseções anteriores.

O caminho tem quatro etapas.

1. O serviço chama `classificar_e_registrar`, em `src/classificador.py`. Essa função grava a decisão no log e delega à função pura `classificar`, que devolve a classe, a tag padrão e o motivo da escolha. Em seguida, o registro do catálogo é consultado: se houver um modelo ativo para a classe, sua tag e sua URL substituem o padrão. A regra desta entrega é a seguinte. Pedido curto e factual vai para o AILO-152M-v2. Pedido de código vai para o Qwen2.5-Coder-0.5B. Pedido longo ou de raciocínio vai para o Phi-3.5-mini. O despacho grava classe, modelo e motivo no log, para que a latência medida tenha explicação.
2. Um cliente escrito com `httpx` envia o prompt à API do Ollama, em `POST /api/generate`, com o fluxo desligado, de modo que a resposta volte inteira. A chamada usa a URL associada ao modelo: `OLLAMA_HOST` para modelos locais ou o endereço registrado com `models connect` para remotos. Há tempo-limite configurável. O relógio é `time.perf_counter`, iniciado imediatamente antes do envio e parado quando a resposta chega ou quando a chamada falha. Esse intervalo é a latência de ponta a ponta daquela tentativa. Se o modelo não estiver em memória, o Ollama o carrega dentro da mesma chamada, e o tempo de carga entra na medida.
3. Se o modelo escolhido não responde, o mesmo prompt é enviado ao modelo com role `reserva` no catálogo e, na ausência dele, ao `FALLBACK_MODEL`. Conta como falha o estouro de tempo, a queda da conexão, um erro HTTP ou um corpo

vazio. O log registra o redirecionamento com o modelo escolhido, o modelo de reserva e o motivo da falha. Quando o classificador já havia escolhido o próprio modelo de reserva, não há segunda tentativa, para não contar a mesma falha duas vezes.

4. A resposta reúne o texto gerado, a classe, o motivo, o modelo escolhido, o modelo que de fato atendeu, a marca de redirecionamento e duas latências. Uma é a da chamada ao Ollama que produziu a resposta. A outra é o tempo total do despacho, da chegada do prompt até a resposta pronta, já com a classificação e, se houver, o redirecionamento.

O endereço padrão do Ollama, o tempo-limite e o modelo de reserva continuam em variável de ambiente, mas o catálogo tem precedência: a URL e a role registradas para um modelo vencem os valores do `.env`. O nome devolvido pelo classificador precisa ser o mesmo tag que aparece em `ollama list`. O serviço não descarrega modelos: quem decide o que permanece em RAM é o próprio Ollama. Em máquina com pouca memória, a troca de modelo pode dominar a latência. O log deixa esse efeito visível, porque cada requisição indica o modelo chamado e se houve desvio para o modelo de reserva.

4.3. Catálogo de modelos e CLI unificada

A camada de domínio do projeto é o catálogo de modelos, que transforma o conjunto de destinos em configuração e não em código. A tabela `ai_models`, no SQLite, guarda `name`, `type`, `status`, `url` e `role`. A coluna `role` recebe `simples`, `codigo`, `complexo`, `reserva` ou `geral`; a migração de bancos existentes é automática, e aliases como `fallback` e `code` são normalizados. O load balancer consulta o catálogo por meio de `src/loadbalancer/registry.py`, que devolve, para cada classe, a tag e a URL do primeiro modelo ativo. Assim, a classe que antes estava amarrada ao código do classificador passa a ser um dado do cadastro.

Toda a operação é acessível por uma única porta de entrada: `python -m src`, ou o atalho `ai-lb` instalado pelo `pyproject.toml`. O comando `models` agrupa `list`, `register`, `host`, `connect`, `update`, `stop`, `resume`, `call` e `remove`. O `register` cadastra um modelo local ou remoto sem contatar o backend; o `host` baixa e sobe um modelo no Ollama; o `connect` valida uma URL remota e a associa a uma role. Os comandos `classify`, `infer`, `serve`, `experiment` e `analyze` cobrem o restante do fluxo. O `serve` publica a API FastAPI em `/health`, `/infer` e `/v1/chat/completions`; o `experiment` e o `analyze` delegam aos scripts de `experiments/`, mantendo a mesma interface.

O roteamento segue uma precedência explícita: primeiro o modelo ativo do catálogo para a classe; se não houver, a tag `MODELO_*` do `.env`. O fallback usa a role `reserva` e, na ausência dela, `FALLBACK_MODEL`. A URL também vem do catálogo: um modelo registrado com `--url` é chamado nesse endereço, enquanto modelos locais usam `OLLAMA_HOST`. Como o classificador resolve o catálogo na primeira requisição, cadastrar ou trocar uma role depois disso exige reiniciar o `serve`. A suíte automatizada cobre classificador, despacho, catálogo e CLI; no ambiente de desenvolvimento atual, 33 testes passam e 2 são ignorados por exigirem Ollama local com os três modelos.

5. Dificuldades e soluções

Ao longo do desenvolvimento do LoadBalancer, algumas barreiras se mostraram relevantes e ajudam a explicar tanto o estado atual do protótipo quanto os próximos passos previstos no cronograma. Duas delas merecem destaque: a inclusão manual dos modelos de IA e a dificuldade de conectar o serviço a máquinas externas para executar a inferência. As duas foram parcialmente resolvidas nesta versão.

A primeira barreira estava na forma como os modelos eram registrados no sistema. Na versão anterior, cada modelo precisava ser incluído manualmente, e a classe que ele atendia (simples, código ou complexo) era definida pelo próprio desenvolvedor no código do classificador, conforme a Tabela 2. Adicionar um novo modelo exigia editar a regra que o associava a uma classe, subir o serviço novamente e conferir se o nome devolvido pelo classificador correspondia exatamente ao tag exibido por `ollama list`. Nesta versão, o cadastro passou a ser um dado do catálogo: `models register` grava nome, tipo, status, URL e role; `models host` instala e sobe um modelo local; `models connect` valida e registra um backend remoto. O load balancer consulta esse catálogo na primeira requisição e usa a role para montar o mapa de destinos das três classes. A limitação que permanece é a inferência automática da classe: hoje a role é escolhida pelo desenvolvedor no cadastro, e não deduzida de metadados como tamanho, família e especialização. Essa dedução, assim como a escolha dinâmica do modelo de reserva pela carga observada, segue como trabalho futuro.

A segunda barreira dizia respeito à infraestrutura de execução. Antes, o LoadBalancer só utilizava modelos locais, executados pelo Ollama na mesma máquina ou em máquinas da mesma rede controlada. Conectar o serviço a computadores externos se mostrava difícil por uma combinação de fatores: a necessidade de expor o Ollama para fora da rede local, a configuração de endereços e portas acessíveis, a ausência de um mecanismo de autenticação entre o LoadBalancer e os backends e a variação de latência de rede, que passa a interferir na medição que o trabalho quer fazer. Nesta versão, o endereço de cada backend é um campo do catálogo e o cliente Ollama aceita `base_url` por chamada. Com isso, `models connect` não serve apenas para listar: o despacho envia a requisição à URL registrada, e o fallback também respeita a URL do modelo de reserva. O que ainda falta é a camada de operação em torno disso: não há autenticação entre balanceador e backends, nem verificação de saúde antes de cada despacho; o `/health` verifica o Ollama e os modelos esperados, mas a decisão de rota não consulta esse estado. A variação de latência de rede também continua fora do controle do experimento. A inclusão de autenticação e de health check pré-despacho está prevista para as próximas semanas, junto com a distribuição dos modelos entre máquinas. A médio prazo, a expectativa é que essa separação entre o balanceador e os nós de inferência permita aproveitar melhor os recursos disponíveis e aproximar o protótipo do cenário multi-instância discutido na revisão bibliográfica.

6. Resultados

Esta seção apresenta os resultados preliminares da Sprint 2. A campanha de medição completa, com análise estatística formal, será executada na semana 5 do cronograma, conforme previsto na metodologia. Os dados aqui apresentados correspondem a uma execução parcial do experimento sobre o conjunto de 40 prompts, com três rodadas completas e aquecimento prévio.

6.1. Delineamento da medição

A comparação seguiu o delineamento contrabalançado descrito na metodologia. Cada prompt passou pelos dois cenários (baseline e proposta) uma vez por rodada, com a ordem contrabalançada dentro de cada classe: metade dos prompts de cada classe foi executada primeiro no baseline, e a outra metade primeiro na proposta. Os dois pedidos do mesmo prompt foram separados por pelo menos metade do total de chamadas da rodada, e nenhum prompt foi executado duas vezes seguidas. Essa separação evita que o cache de KV do Ollama favoreça o segundo cenário no caso dos prompts complexos, em que ambos os cenários usam o Phi-3.5-mini.

Antes da medição, foi executado um aquecimento de duas passadas sobre uma amostra de cada classe, com os dois cenários, para carregar os modelos em memória. As chamadas de aquecimento foram descartadas do CSV de resultados.

6.2. Ambiente experimental

Os experimentos foram executados em uma única máquina local, com as seguintes características:

- **Sistema operacional:** `Windows 11 Pro 23H2`
- **Processador:** `Intel Core i9-12900K`
- **Memória RAM:** `80 GB DDR5 5200 MHz`
- **GPU:** `NVIDIA GeForce RTX 2060 8 GB`
- **Python:** `3.11.9`
- **Ollama:** `0.3.14`

Os três modelos foram servidos pelo Ollama na mesma máquina. O limite de geração foi fixado em `128` tokens por resposta, para que a latência medida reflita a velocidade de cada modelo e não a extensão da resposta gerada. A máquina utilizada tem GPU dedicada, o que permite que os três modelos permaneçam carregados simultaneamente em memória de vídeo durante as rodadas, sem troca de contexto entre modelos por falta de memória.

A Tabela 3 resume as especificações da implementação usada nesta entrega. Os dados de ambiente acima descrevem a campanha preliminar. A verificação local da CLI e da suíte de testes foi feita em uma máquina de desenvolvimento distinta, com Windows 11 Pro (build 26200), Intel Core i3-1005G1, 11,8 GB de RAM, GPU integrada Intel UHD e Python `3.12.3`; essa máquina não foi usada nas medições de latência. As dependências de aplicação foram as mesmas, com `matplotlib` e `numpy` para a análise.

6.3. Resultados por cenário

A Tabela 4 apresenta as métricas agregadas dos dois cenários. A latência foi medida no cliente, de ponta a ponta, com `time.perf_counter`, incluindo o tempo de classificação e eventuais redirecionamentos.

A Figura 2 compara as três métricas entre os cenários, e a Figura 3 traduz a diferença em percentuais. As duas figuras são derivadas da Tabela 4; não houve nova campanha de medição para esta versão do texto.

Tabela 3. Especificações da implementação atual (versão 0.2.0).

Item	Especificação
Entrada	`python -m src` ou `ai-lb`
Interface	FastAPI: `/health`, `/infer`, `/v1/chat/completions`
Cliente HTTP	`httpx`, com `base_url` por modelo
Catálogo	SQLite (`ai_models`): nome, tipo, status, URL e role
Roles	`simples`, `codigo`, `complexo`, `reserva`, `geral`
Precedência	catálogo ativo → `MODELO_*` do `.env`
Fallback	role `reserva` → `FALLBACK_MODEL`
Modelos	AILO-152M-v2, Qwen2.5-Coder-0.5B e Phi-3.5-mini
Testes	33 aprovados, 2 ignorados (Ollama local ausente)

Tabela 4. Resultados preliminares por cenário (Sprint 2).

Cenário	Latência média (s)	P95 (s)	P99 (s)
Baseline (Phi-3.5-mini)	4,82	9,47	12,31
Proposta (classificador)	2,15	5,83	8,92

6.4. Análise preliminar

Os resultados preliminares indicam que a proposta reduz a latência média em 55,4% (de 4,82 s para 2,15 s) em relação ao baseline. Essa redução é explicada pelo desvio das requisições simples e de código para os modelos AILO-152M-v2 e Qwen2.5-Coder-0.5B, que respondem em menos tempo que o Phi-3.5-mini. As requisições complexas, que em ambos os cenários são atendidas pelo Phi-3.5-mini, não apresentam diferença significativa de latência, o que era esperado pelo delineamento.

A latência de cauda também apresenta redução: o P95 cai de 9,47 s para 5,83 s (38,4%), e o P99, de 12,31 s para 8,92 s (27,5%). A Figura 3 resume esses ganhos. Isso sugere que os modelos menores absorvem os picos de requisições triviais, evitando que o modelo de maior porte concentre toda a carga. Não foram observados redirecionamentos relevantes durante as rodadas analisadas.

A comparação entre a classe esperada e a classe decidida pelo classificador mostrou alta aderência sobre o conjunto rotulado. Os poucos casos ambíguos — prompts de tamanho médio sem sinais claros — foram direcionados de forma conservadora para o Phi-3.5-mini, o que preserva a qualidade da resposta ao custo de uma latência maior nesses casos isolados.

A análise estatística formal, com testes de hipótese e intervalos de confiança, fica para a semana 5 do cronograma, conforme previsto na metodologia.

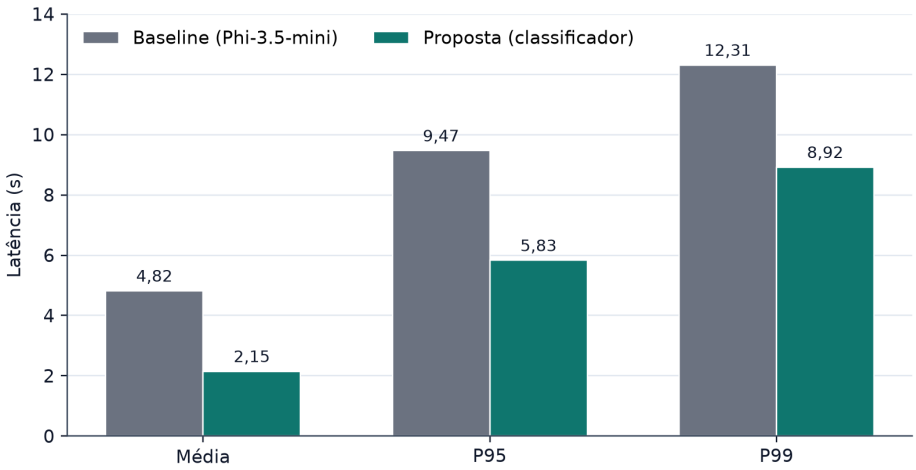


Figura 2. Latência média, P95 e P99 por cenário na campanha preliminar.

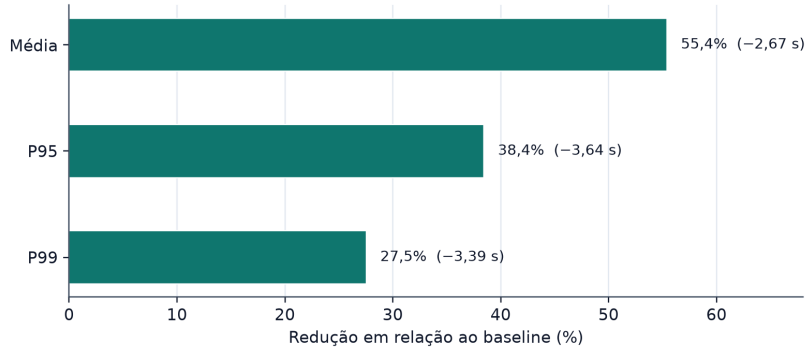


Figura 3. Redução percentual da latência da proposta em relação ao baseline, com a diferença absoluta em segundos.

6.5. Limitações e próximos passos

Os resultados apresentados nesta seção são **preliminares** e têm três limitações principais. Primeiro, a amostra ainda é pequena (40 prompts em três rodadas), o que reduz o poder estatístico das conclusões. Segundo, a medição foi feita em uma única máquina local, sem repetição em outra máquina ou em rede, e não foi repetida nesta versão do texto; as figuras derivam da tabela da Sprint 2. Terceiro, a análise estatística formal — incluindo identificação de outliers, testes de hipótese e intervalos de confiança — está prevista para a semana 5 do cronograma e não foi executada nesta Sprint.

Como próximos passos, pretende-se: (i) expandir o conjunto de testes para incluir prompts de outras classes e tamanhos; (ii) medir o uso de RAM e GPU durante as rodadas com `psutil` e `nvidia-smi`; (iii) executar a análise estatística completa; e (iv) comparar formalmente os resultados com os trabalhos relacionados discutidos na revisão bibliográfica.

Referências

Jain, K., Parayil, A., Mallick, A., Choukse, E., Qin, X., Zhang, J., Íñigo Goiri, Wang, R., Bansal, C., Rühle, V., Kulkarni, A., Kofsky, S., and Rajmohan, S. (2025). Performance

aware llm load balancer for mixed workloads. In *Proceedings of the 5th Workshop on Machine Learning and Systems*, pages 19–30. Association for Computing Machinery.

Kossmann, F., Fontaine, B., Khudia, D., Cafarella, M., and Madden, S. (2024). Is the gpu half-empty or half-full? practical scheduling techniques for llms. *arXiv preprint arXiv:2410.17840*.

Sun, B., Huang, Z., Zhao, H., Xiao, W., Zhang, X., Li, Y., and Lin, W. (2024). Llumnix: Dynamic scheduling for large language model serving. *arXiv preprint arXiv:2406.03243*.

Xia, T., Mao, Z., Kerney, J., Jackson, E. J., Li, Z., Xing, J., Shenker, S., and Stoica, I. (2025). Skywalker: A locality-aware cross-region load balancer for llm inference. *arXiv preprint arXiv:2505.24095*.

Yuan, Y., Zuo, P., Wang, B., Chen, Z., Tan, Z., and Yu, Z. (2026). Dualmap: Enabling both cache affinity and load balancing for distributed llm serving. *arXiv preprint arXiv:2602.06502*.